Assignment 1: Design and Implementation of a Single-Server Key-Value Store

Ashmitha Ashokkumar, Jyothsna Mysore Santhosh Kumar

1. Design Decisions and Justification

Language and server: We used Python with Flask for routing, served by Waitress with 8 worker threads. Flask keeps the API code short and matches the provided benchmark's JSON API. Flask's built-in server is meant only for development. The GIL limits CPU parallelism, but this workload is dominated by network and disk waits, during which threads overlap.

API and errors: The server supports GET, PUT and DELETE on /<key>. POST is accepted as PUT, since the benchmark uses it, and GET / is a health check. PUT returns 201 for a new key and 200 for an update. Errors are returned as JSON: 404 missing key, 400 malformed body or key over 256 characters, 405 unsupported method, 413 body over 1 MiB, 500 unexpected error.

Concurrency: Data is kept in an in-memory dictionary guarded by a single lock. DEL checks for and removes a key in the same critical section, so of many simultaneous deletes exactly one succeeds. Concurrent writes to one key are applied one at a time (last writer wins).

Persistence: Each PUT/DEL is appended to a write-ahead log and flushed before memory changes or the client is answered, so acknowledged writes survive a process crash. Every 30 s a background thread writes the dictionary to a temporary file, atomically renames it to snapshot.json, starts a new log and deletes older ones. On startup the server loads the snapshot and replays newer logs. An optional --fsync flag adds power-loss durability at a throughput cost.

Logging: An after-request hook records every operation, including failed ones, with a timestamp, operation, key, status code, client and latency. Entries go to a rotating log file.

2. Challenges and Solutions

- **Provided benchmark:** It writes key_0, key_2, … but reads key_1, key_3, …, so a correct server returns 404 for all 150 reads. It also counts 600 operations while sending 300. We kept the required 404s and wrote a corrected benchmark that reads back its own keys.
- **Log and memory order:** Writing the log and updating memory under the same lock makes the log order identical to the apply order, so recovery rebuilds the same state.
- **Snapshots without blocking:** Only the dictionary copy and the log switch hold the lock; the disk write happens outside it. Old logs are deleted only after the new snapshot is in place.
- **Half-written records:** A crash mid-write can leave a partial last log line. Recovery skips it, which is safe because that write was never acknowledged, and each start writes to a new log file.

3. Testing and Results

All 17 automated tests pass. They cover every operation, the error cases, concurrent same-key access, clean restart, crash recovery, malformed log lines and snapshots. With 3 client threads and 300 operations per phase on a local machine, writes reached 792 ops/s (avg 3.77 ms, p99 8.33 ms) and reads 963 ops/s (avg 3.11 ms, p99 5.28 ms). There were no errors or value mismatches.

4. Assumptions

All data fits in one machine's memory. Keys are at most 256 characters and request bodies at most 1 MiB. The durability target is a process crash; power-loss safety requires --fsync. Last-writer-wins is sufficient, with no transactions. The server runs on a trusted network without authentication or TLS.

5. Potential Improvements

- A readers-writer or per-key lock, so reads and unrelated writes run in parallel.
- Group commit, to make --fsync cheaper and Compare-and-swap and key expiry (TTL).
- Partitioning with consistent hashing, and replication across servers.
- Authentication, TLS and monitoring.